

LAPORAN UAS

Nama : Widiono
NIM : 1103210060
Kelas : TK-45-G09

Chapter 1

Pada Chapter 1 ini menjelaskan tentang pengistallan Ros, pertama-tama kita buka google lalu salin link tersebut <https://wiki.ros.org/noetic/Installation/Ubuntu>, setelah itu kita ikuti langkah langkah yang ada pada Ros tersebut.

Chapter 2

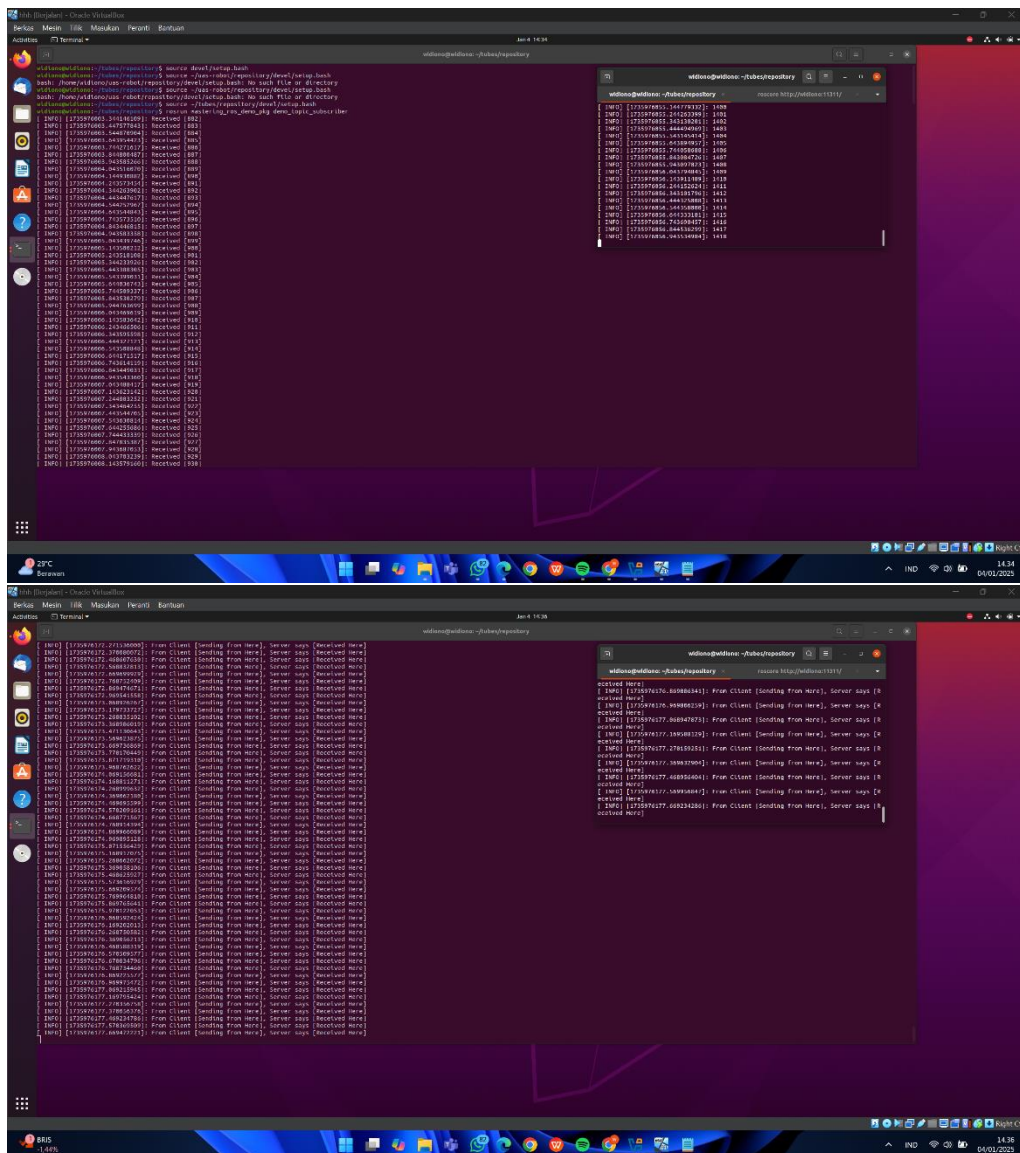
Cara menginstall-nya:

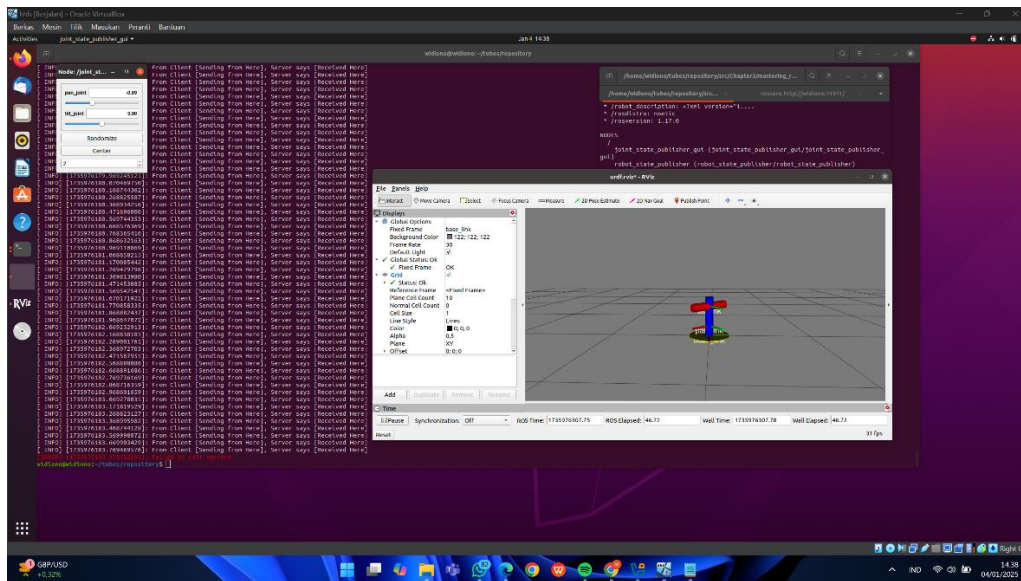
Pastikan ros-noetic sudah terinstall dan rosdep, dan ada beberapa yang harus diinputkan pada terminal:

- Sudo apt update
- mkdir (masukan file nama yang diinginkan)
- cd (masukan file yang telah dibuat tadi)
- git clone <https://github.com/PacktPublishing/Mastering-ROS-for-Robotics-Programming-Third> edition.git repository
- jika tidak bisa, bisa memakai ssh, “git clone git@github.com:PacktPublishing/Mastering-ROS-for-Robotics-Programming-Third edition.git repository”
- cd repository
- cd repository
- mkdir src
- mv Chapter2 src/
- rosdep install --from-paths src --ignore-src -r -y
- catkin_make
- rosrn mastering_ros_demo_pkg demo_topic_subscriber
- rosrn mastering_ros_demo_pkg demo_msg_subscriber
- rosrn mastering_ros_demo_pkg demo_action_client
- rosrn mastering_ros_demo_pkg demo_service_client

(Terminal Baru)

- roscore
- source ~/tubes/repository/devel/setup.bash
- rosrn mastering_ros_demo_pkg demo_topic_publisher
- rosrn mastering_ros_demo_pkg demo_msg_publisher
- rosrn mastering_ros_demo_pkg demo_action_server
- rosrn mastering_ros_demo_pkg demo_service_server





Kesimpulan Chapter 2:

Pada kesimpulan ini menjelaskan langkah-langkah instalasi dan konfigurasi untuk menjalankan proyek Mastering ROS for Robotics Programming. Rangkaian perintah mencakup instalasi dependensi dasar ROS Noetic, setup ROS workspace menggunakan *catkin*, serta pengujian aplikasi demo publisher-subscriber. Beberapa langkah penting melibatkan pembaruan repositori sistem (`sudo apt update`), instalasi alat pendukung seperti `roscdep` dan `catkin-tools`, serta pengaturan variabel lingkungan dengan perintah `source`.

Salah satu hal penting adalah penggunaan `source ~/Downloads/repository/devel/setup.bash` di setiap terminal baru. Ini memastikan ROS dapat menemukan paket yang di-build dalam workspace. Jika dilewatkan, muncul kesalahan seperti "[rospack] Error: package 'mastering_ros_demo_pkg' not found", karena environment ROS tidak mengetahui lokasi build. Selain itu, perhatian terhadap sensitivitas huruf besar/kecil dalam penamaan direktori seperti Chapter2 menjadi penting, sebab kesalahan kecil dalam penamaan dapat mengakibatkan kegagalan build atau runtime.

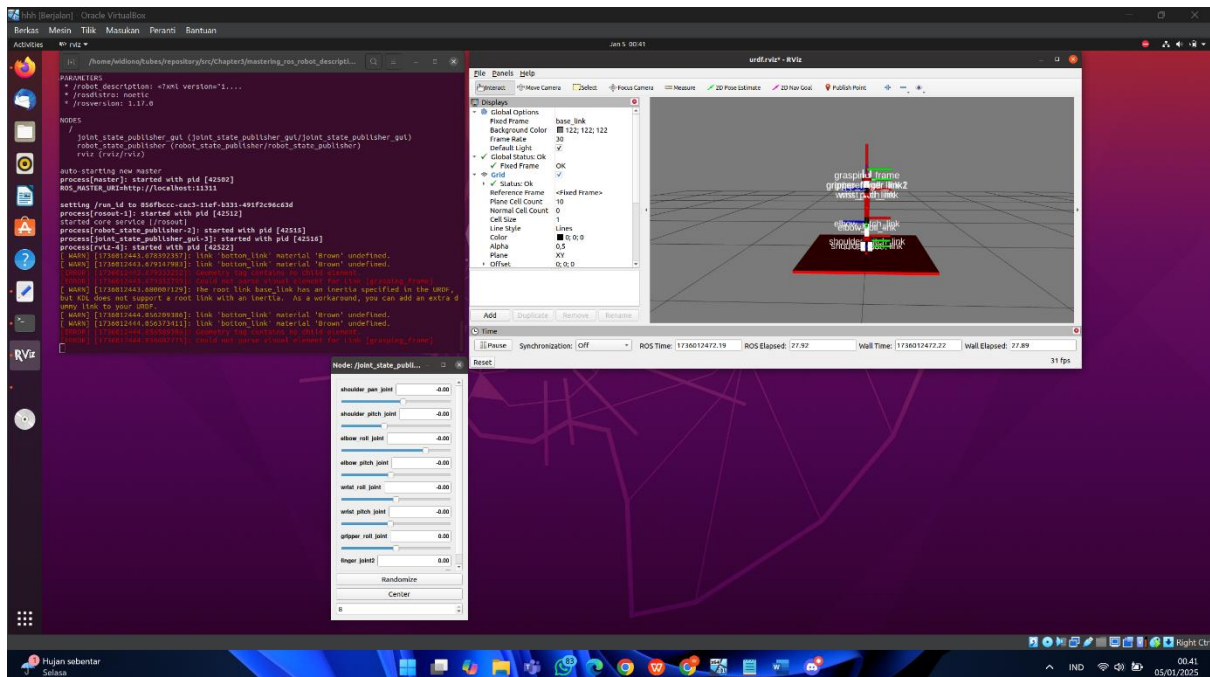
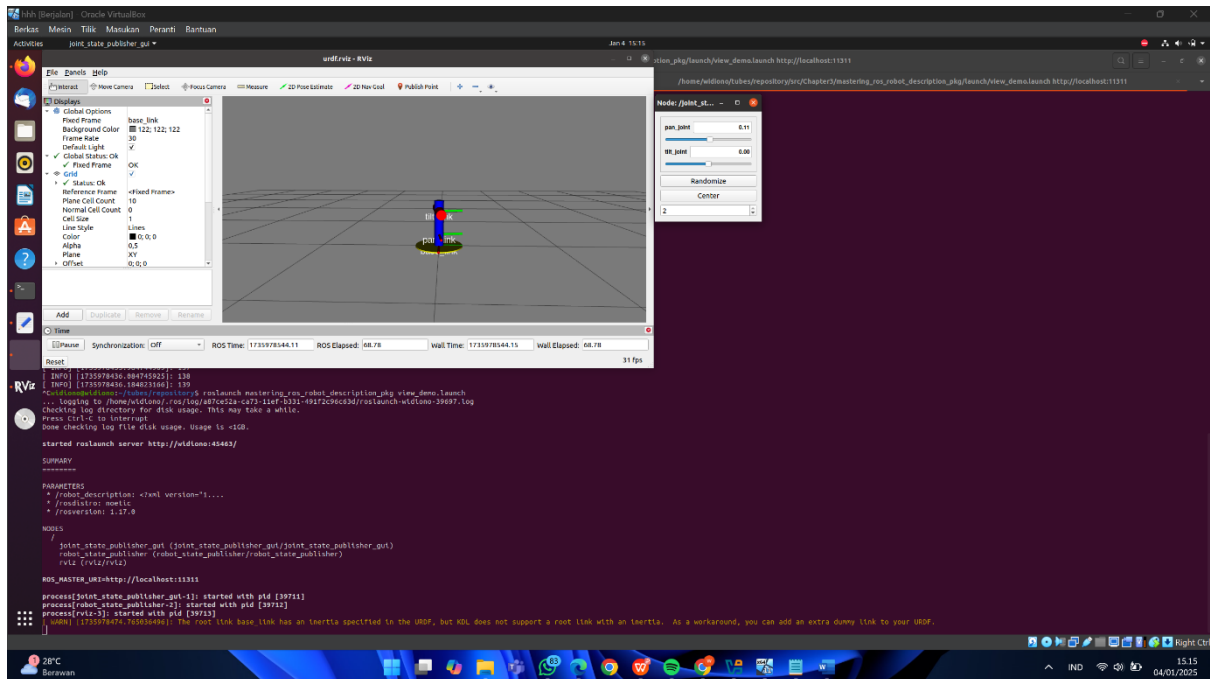
Langkah-langkah pemulihan, seperti membersihkan cache apt (`sudo apt clean`) dan memperbaiki konfigurasi `roscdep` jika terjadi kesalahan, menunjukkan solusi praktis untuk kendala umum. Secara keseluruhan, prosedur ini mencerminkan alur kerja standar dalam pengembangan proyek berbasis ROS, dengan penekanan pada pengaturan lingkungan yang tepat dan instalasi dependensi agar proses build dan eksekusi berjalan lancar.

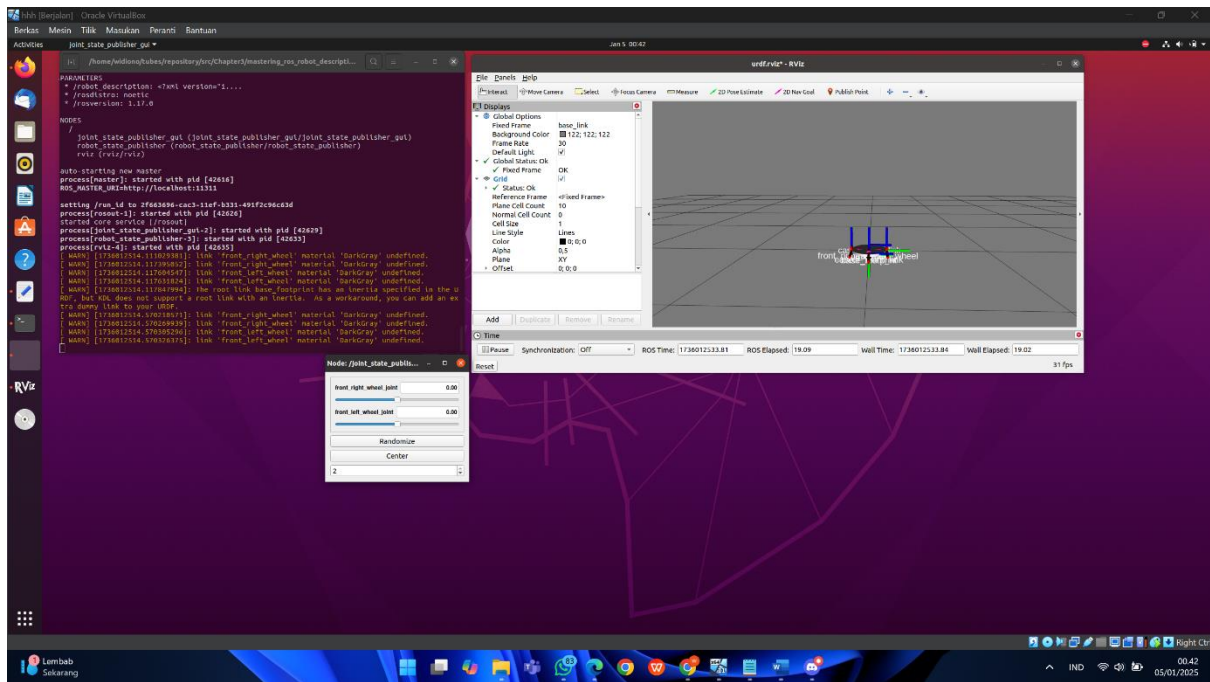
Chapter 3

Pada chapter 3 ini ada beberapa yang harus diinputkan pada terminal :

- `Source/opt/ros/noetic/setup.bash`
- `Sudo apt update`
- `Sudo apt install python3-rosdep python3-catkin-tools ros-noetic-xacro ros-noetic-joint-state-publisher-gui ros-noetic-rviz`
- `Sudo apt install ros-noetic-rviz`
- `Rosdep update`
- `cd ~/tubes(sesuai nama file yang dimasukan)/repository`
- `ls`
- `mv Chapter3 src/`
- `catkin_make`
- `source devel/setup.bash`
- `roslaunch mastering_ros_robot_description_pkg view_demo.launch`
- `source devel/setup.bash`
- `roslaunch mastering_ros_robot_description_pkg view_arm.launch`
- `source devel/setup.bash`
- `roslaunch mastering_ros_robot_description_pkg view_mobile_robot.launch`

Hasil simulasi Chapter 3 :





Kesimpulan Chapter 3:

Dari hasil Chapter 3 ini, perintah yang digunakan untuk menjalankan file URDF melalui ROS berhasil memvisualisasikan model robot pada RViz. Ini menandakan bahwa workspace ROS sudah dikonfigurasi dengan benar menggunakan `catkin_make` dan dependensi yang diperlukan sudah terpasang. Dengan menjalankan `roslaunch mastering_ros_robot_description_pkg view_demo.launch`, node-node seperti `joint_state_publisher_gui` dan `robot_state_publisher` diinisialisasi untuk mempublikasikan data robot, sehingga model robot dapat dimanipulasi (misalnya, menggunakan slider untuk mengatur posisi joint).

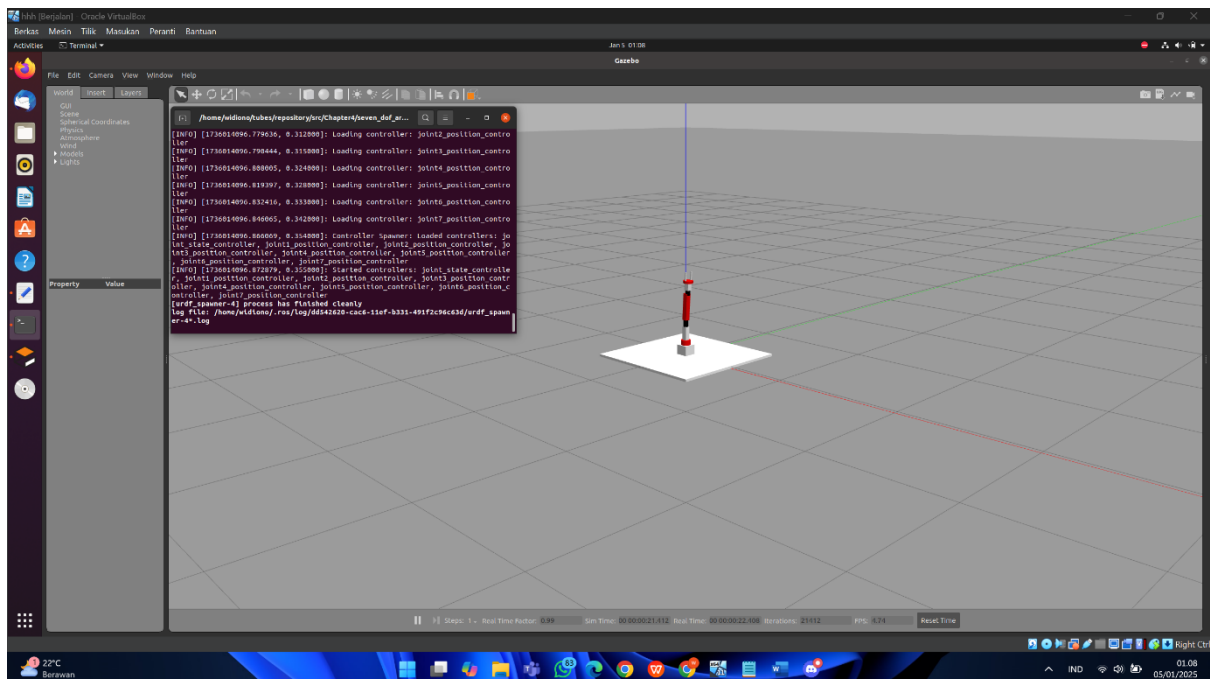
Namun, ada peringatan tentang `base_link` yang memiliki properti inertia tetapi KDL tidak mendukung root link dengan inertia. Ini bukan masalah kritis dan hanya memengaruhi simulasi jika inertia benar-benar digunakan untuk perhitungan fisika. Untuk mengatasi peringatan ini, Anda dapat menambahkan dummy link sebagai root link di file URDF, seperti yang disarankan dalam pesan peringatan. Selain itu, keberhasilan visualisasi model menunjukkan bahwa sistem ROS bekerja dengan baik dalam mengintegrasikan URDF dan RViz untuk memvalidasi desain robot.

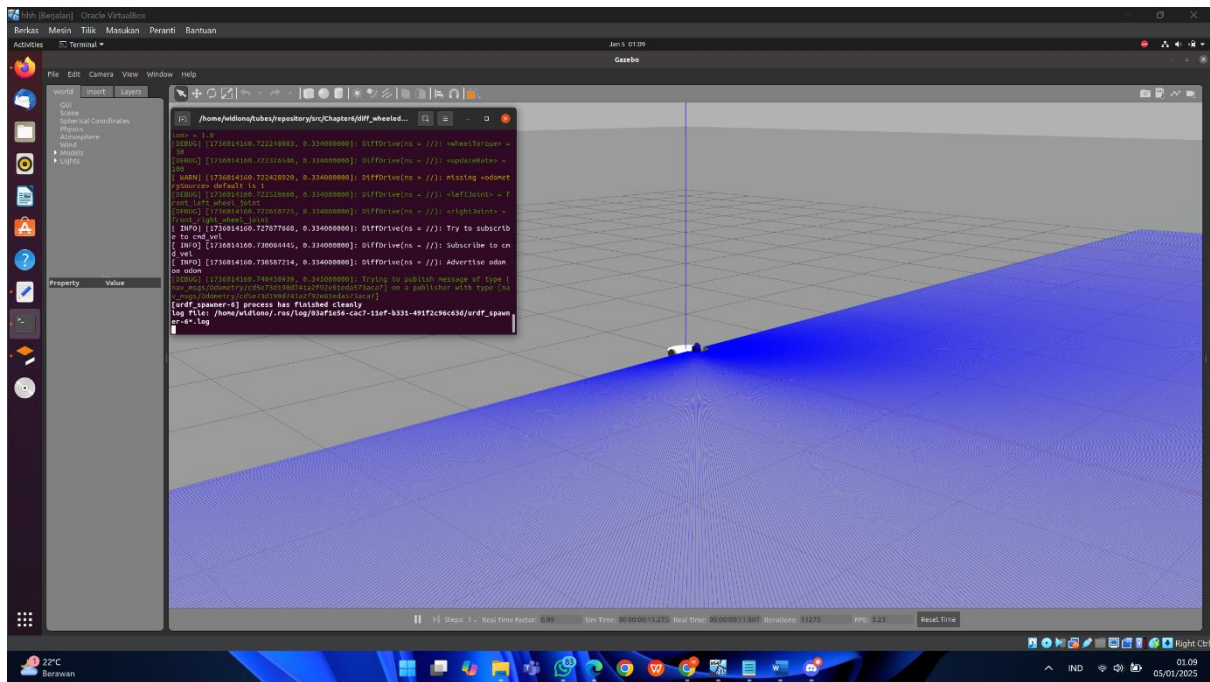
Chapter 4

Pada chapter 4 ini ada beberapa yang harus diinputkan pada terminal :

- `mv Chapter4 src/`
- `mv Chapter6 src/`
- `catkin_make`
- `source devel/setup.bash`
- `roslaunch seven_dof_arm_gazebo seven_dof_arm_gazebo_control.launch`
- `rostopic pub /seven_dof_arm/joint4_position_controller/command std_msgs/Float64 "data: 1.0"`
- `roslaunch diff_wheeled_robot_gazebo diff_wheeled_gazebo_full.launch`
- `roslaunch diff_wheeled_robot_control keyboard_teleop.launch`
- `rviz`

Hasil Simulasi Chapter 4:





Kesimpulan dari Chapter 4 :

Dari kedua tangkapan layar, simulasi robot menggunakan Gazebo berhasil dijalankan untuk dua jenis robot: manipulator lengan robotik 7-DOF dan robot beroda diferensial. Pada simulasi pertama, lengan robotik 7-DOF divisualisasikan dengan pengontrol posisi joint berhasil dimuat. Penggunaan perintah `rostopic pub` memungkinkan pengujian pergerakan individu joint dengan mengirimkan pesan posisi ke salah satu pengontrol, menunjukkan bahwa pengaturan kontrol berbasis ROS bekerja sebagaimana mestinya. Ini membuktikan bahwa integrasi antara model URDF, plugin Gazebo, dan node kontrol sudah berjalan baik.

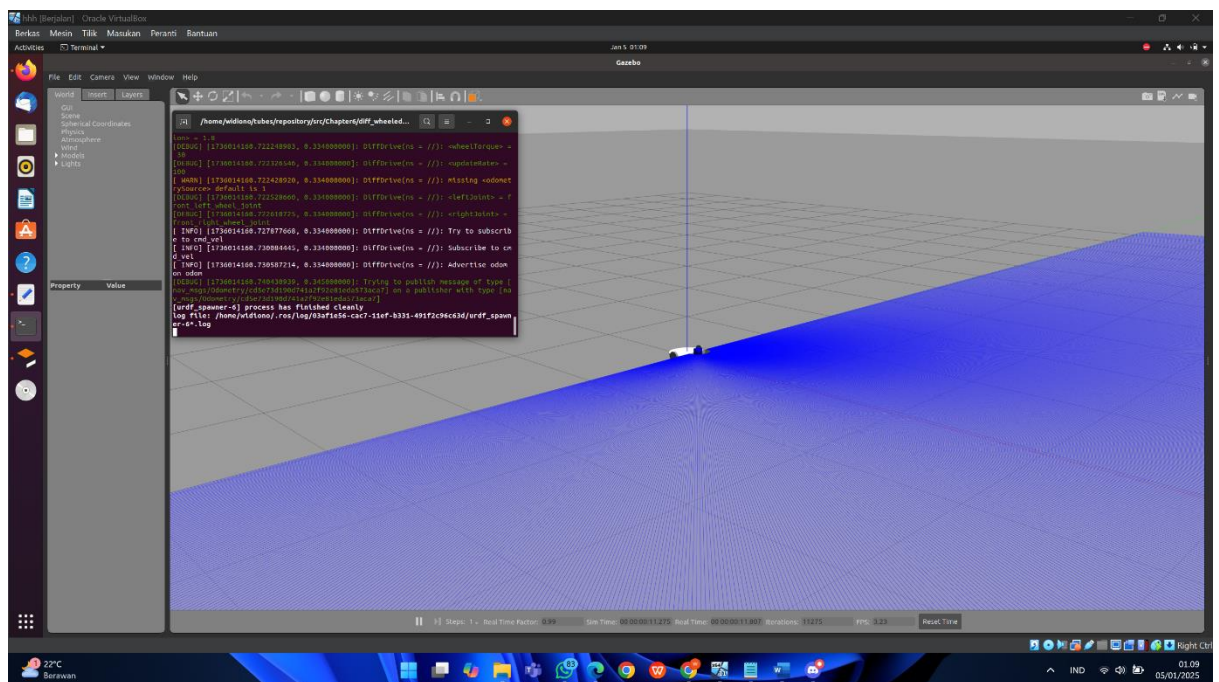
Pada simulasi kedua, robot beroda diferensial divisualisasikan di lingkungan Gazebo dengan pengontrol penuh. File `diff_wheeled_gazebo_full.launch` memuat model robot, plugin kontroler, dan node navigasi. Namun, peringatan terkait pesan `odom` menunjukkan bahwa ada kemungkinan kesalahan dalam komunikasi data odometri atau pengaturan plugin. Untuk mengontrol pergerakan robot, perintah `keyboard_teleop.launch` digunakan, memungkinkan manipulasi manual robot. Secara keseluruhan, kedua simulasi ini menunjukkan kemampuan ROS dan Gazebo untuk menjalankan simulasi robot yang kompleks dengan pengontrol khusus, meskipun masih diperlukan debugging lebih lanjut untuk memperbaiki beberapa peringatan kecil yang muncul.

Chapter 6

Pada Chapter 6 ini ada beberapa yang harus diinputkan terlebih dahulu ke terminal:

- `catkin_make`
- `source devel/setup.bash`
- `roslaunch diff_wheeled_robot_gazebo diff_wheeled_gazebo_full.launch`
- `roslaunch diff_wheeled_robot_gazebo gmapping.launch`
- `roslaunch diff_wheeled_robot_control keyboard_teleop.launch`
- `rviz`
- `roslaunch diff_wheeled_robot_gazebo diff_wheeled_gazebo_full.launch`
- `roslaunch diff_wheeled_robot_gazebo amcl.launch`
- `rviz`

Hasil Simulasi Chapter 6:



Kesimpulan dari Chapter 6:

Rangkaian gambar dan perintah yang diberikan menggambarkan proses simulasi robot roda diferensial menggunakan ROS dan Gazebo. Proses dimulai dengan kompilasi *workspace* ROS menggunakan `catkin_make`, diikuti dengan pengaturan *environment* menggunakan `source devel/setup.bash`. Langkah selanjutnya adalah meluncurkan simulasi robot di Gazebo menggunakan perintah `roslaunch diff_wheeled_robot_gazebo diff_wheeled_gazebo_full.launch`. Pada jendela Gazebo, terlihat model robot yang sederhana beserta *output log* yang memberikan informasi tentang konfigurasi dan beberapa peringatan terkait dengan sumber *odometry* yang hilang. Kemudian, dilakukan upaya untuk membuat peta menggunakan *node* `gmapping`, yang dilanjutkan dengan peluncuran *node* `teleop` untuk kendali robot secara manual dan aplikasi visualisasi `rviz`. Pada awalnya `rviz` hanya menunjukkan tampilan *grid*, yang mengindikasikan bahwa proses *mapping* belum berhasil atau data belum diterima `rviz`.

Selanjutnya, serangkaian perintah diluncurkan kembali, di mana simulasi Gazebo dijalankan ulang, diikuti dengan peluncuran *node* `amcl` untuk *localization* dan kembali meluncurkan `rviz`. Hal ini menandakan bahwa pengguna mencoba untuk mengimplementasikan algoritma *localization* setelah *mapping*. Pengulangan peluncuran simulasi Gazebo mengindikasikan adanya kemungkinan kendala atau kesalahan yang terjadi dalam proses sebelumnya. Secara keseluruhan, rangkaian perintah menunjukkan upaya untuk mensimulasikan robot, melakukan *mapping*, kendali, dan *localization*, dengan adanya indikasi masalah terkait dengan *odometry* dan proses *mapping*. Perlu dilakukan analisis lebih lanjut pada pesan-pesan *log* dan konfigurasi sistem untuk mengidentifikasi dan mengatasi masalah yang ada, agar implementasi *mapping* dan *localization* berhasil.